



SETUP GUIDE

Prometheus + Grafana for Model Monitoring

SECTION 1: OVERVIEW

What we build:

- A local monitoring stack using Docker Compose
- Prometheus scraping metrics from a simulated ML inference service
- Grafana dashboards visualising inference latency, throughput, predictions
- An alerting rule that fires when P99 latency spikes

SECTION 2: PREREQUISITES

Required software:

- Docker Desktop (installed in Week 4)
- Docker Compose v2+ (bundled with Docker Desktop)
- Python 3.8+ (for the metrics exporter script)
- A web browser (for Prometheus and Grafana UIs)
- ~500MB disk space for Docker images

Ports needed:

- 8000 — ML metrics exporter
- 9090 — Prometheus
- 3000 — Grafana

IMPORTANT: If you still have Triton running from Week 8, stop it first:

```
docker ps          # Check if tritonserver is running
docker stop <container_name> # Stop it — both use port 8000
```

Verify Docker is running:

```
docker --version
docker compose version
```

SECTION 3: PROJECT STRUCTURE

Create this folder structure:



```
week9-monitoring/  
├─ docker-compose.yml  
└─ prometheus/
```

SECTION 4: FILE CONTENTS

FILE: docker-compose.yml

services:

ml-exporter:

build: ./ml-exporter

container_name: ml-metrics-exporter

ports:

- "8000:8000"

networks:

- monitoring

prometheus:

image: prom/prometheus:v2.51.0

container_name: prometheus

ports:

- "9090:9090"

volumes:

- ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro

- ./prometheus/alerts.yml:/etc/prometheus/alerts.yml:ro

- prometheus-data:/prometheus

command:

- "--config.file=/etc/prometheus/prometheus.yml"

- "--storage.tsdb.retention.time=7d"

depends_on:

- ml-exporter

networks:

- monitoring

grafana:

image: grafana/grafana:10.4.0

container_name: grafana

ports:

- "3000:3000"



environment:

- GF_SECURITY_ADMIN_USER=admin
- GF_SECURITY_ADMIN_PASSWORD=admin123

volumes:

- ./grafana/provisioning:/etc/grafana/provisioning:ro
- grafana-data:/var/lib/grafana

depends_on:

- prometheus

networks:

- monitoring

networks:

monitoring:

driver: bridge

volumes:

prometheus-data:

grafana-data:

FILE: prometheus/prometheus.yml

global:

scrape_interval: 15s

evaluation_interval: 15s

rule_files:

- "alerts.yml"

scrape_configs:

- job_name: "prometheus"

static_configs:

- targets: ["localhost:9090"]

- job_name: "ml-inference"

static_configs:

- targets: ["ml-exporter:8000"]

metrics_path: "/metrics"

scrape_interval: 15s

FILE: prometheus/alerts.yml

groups:

- name: ml_inference_alerts

rules:

- alert: HighP99Latency

expr: histogram_quantile(0.99, rate(inference_latency_seconds_bucket[5m])) > 0.2

for: 2m

labels:

severity: warning

annotations:

summary: "P99 inference latency is above 200ms"

description: "P99 latency is {{ \$value }}s for the last 5 minutes."

- alert: HighErrorRate

expr: rate(inference_errors_total[5m]) / rate(inference_requests_total[5m]) > 0.05

for: 2m

labels:

severity: critical

annotations:

summary: "Inference error rate above 5%"

description: "Error rate is {{ \$value | humanizePercentage }}."

- alert: PredictionImbalance

expr: |

max(inference_predictions_total) / sum(inference_predictions_total) > 0.95

for: 5m

labels:

severity: warning

annotations:

summary: "Single prediction class dominates (>95%)"

description: "Model may be stuck or receiving biased input."

FILE: grafana/provisioning/datasources/prometheus.yml

apiVersion: 1

datasources:

- name: Prometheus

type: prometheus

access: proxy

url: http://prometheus:9090

isDefault: true

editable: true

FILE: ml-exporter/requirements.txt



```
prometheus-client==0.20.0  
flask==3.0.2
```

FILE: ml-exporter/Dockerfile

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app.py .
```

```
EXPOSE 8000
```

```
CMD ["python", "app.py"]
```

FILE: ml-exporter/app.py

```
"""
```

Simulated ML Inference Metrics Exporter

Exposes Prometheus metrics that simulate a production ML inference service.
Generates realistic latency distributions, request counts, and prediction
class distributions for monitoring demonstration.

```
"""
```

```
import time
```

```
import random
```

```
import threading
```

```
from prometheus_client import (
```

```
    Counter,
```

```
    Histogram,
```

```
    Gauge,
```

```
    generate_latest,
```

```
    CONTENT_TYPE_LATEST,
```

```
)
```

```
from flask import Flask, Response
```

```
app = Flask(__name__)
```

```
# — Metric Definitions —————
```

```
# Request counter
inference_requests_total = Counter(
    "inference_requests_total",
    "Total inference requests served",
    ["model_name", "model_version"],
)

# Error counter
inference_errors_total = Counter(
    "inference_errors_total",
    "Total inference errors",
    ["model_name", "error_type"],
)

# Latency histogram (buckets in seconds)
inference_latency_seconds = Histogram(
    "inference_latency_seconds",
    "Inference latency in seconds",
    ["model_name", "model_version"],
    buckets=[0.01, 0.025, 0.05, 0.075, 0.1, 0.15, 0.2, 0.3, 0.5, 1.0, 2.0],
)

# Prediction class counter
inference_predictions_total = Counter(
    "inference_predictions_total",
    "Predictions by class",
    ["model_name", "predicted_class"],
)

# Confidence gauge (average over recent window)
inference_confidence_avg = Gauge(
    "inference_confidence_avg",
    "Average prediction confidence (recent window)",
    ["model_name"],
)

# GPU utilization gauge
gpu_utilization_percent = Gauge(
    "gpu_utilization_percent",
    "Simulated GPU utilization",
    ["gpu_id"],
```

)

Batch size gauge

```
inference_batch_size = Gauge(  
    "inference_batch_size",  
    "Current inference batch size",  
    ["model_name"],  
)
```

——— Simulation Logic —————

MODEL_NAME = "sentiment_classifier"

MODEL_VERSION = "v2.1"

CLASSES = ["positive", "negative", "neutral"]

def simulate_normal_traffic():

"""Simulate normal inference traffic with realistic distributions."""

Normal latency: mostly fast with occasional slow requests

latency = random.gauss(0.045, 0.015)

latency = max(0.005, latency) # Floor at 5ms

Occasionally inject a slow request (simulates cold start or GC)

if random.random() < 0.02:

latency = random.uniform(0.3, 0.8)

Record metrics

```
inference_requests_total.labels(  
    model_name=MODEL_NAME, model_version=MODEL_VERSION  
)inc()
```

```
inference_latency_seconds.labels(  
    model_name=MODEL_NAME, model_version=MODEL_VERSION  
)observe(latency)
```

Prediction distribution (slightly skewed toward positive)

weights = [0.45, 0.30, 0.25]

predicted_class = random.choices(CLASSES, weights=weights, k=1)[0]

```
inference_predictions_total.labels(  
    model_name=MODEL_NAME, predicted_class=predicted_class  
)inc()
```

Confidence score

```
confidence = random.gauss(0.82, 0.08)
confidence = max(0.5, min(1.0, confidence))
inference_confidence_avg.labels(model_name=MODEL_NAME).set(confidence)

# Simulate errors (1% rate normally)
if random.random() < 0.01:
    error_type = random.choice(["timeout", "oom", "validation"])
    inference_errors_total.labels(
        model_name=MODEL_NAME, error_type=error_type
    ).inc()

def simulate_gpu_metrics():
    """Simulate GPU utilization fluctuating around 65%."""
    utilization = random.gauss(65, 10)
    utilization = max(0, min(100, utilization))
    gpu_utilization_percent.labels(gpu_id="gpu-0").set(utilization)

def simulate_batch_size():
    """Simulate varying batch sizes."""
    batch = random.choice([1, 1, 1, 2, 4, 8, 8, 16])
    inference_batch_size.labels(model_name=MODEL_NAME).set(batch)

def background_simulation():
    """Run continuous simulation in background thread."""
    while True:
        # Simulate 5-15 requests per second
        requests_this_second = random.randint(5, 15)
        for _ in range(requests_this_second):
            simulate_normal_traffic()

        simulate_gpu_metrics()
        simulate_batch_size()
        time.sleep(1)

# — Flask Endpoints —————

@app.route("/metrics")
def metrics():
    """Prometheus metrics endpoint."""
    return Response(generate_latest(), mimetype=CONTENT_TYPE_LATEST)

@app.route("/health")
def health():
```

```
"""Health check endpoint."""
```

```
return {"status": "healthy", "model": MODEL_NAME, "version": MODEL_VERSION}
```

```
# ——— Start —————
```

```
if __name__ == "__main__":
```

```
    # Start background simulation thread
```

```
    sim_thread = threading.Thread(target=background_simulation, daemon=True)
```

```
    sim_thread.start()
```

```
    print(f"ML Metrics Exporter starting on port 8000...")
```

```
    print(f"Model: {MODEL_NAME} {MODEL_VERSION}")
```

```
    print(f"Metrics at: http://localhost:8000/metrics")
```

```
    app.run(host="0.0.0.0", port=8000)
```

SECTION 5: STEP-BY-STEP SETUP

STEP 1: Create project folder

```
mkdir week9-monitoring
```

```
cd week9-monitoring
```

STEP 2: Create all files as shown in Section 4 above

(Copy each file into the appropriate path)

STEP 3: Build and start all services

```
docker compose up --build -d
```

Expected output:

```
[+] Building ml-exporter...
```

```
[+] Running 4/4
```

```
✓ Network week9-monitoring_monitoring Created
```

```
✓ Container ml-metrics-exporter Started
```

```
✓ Container prometheus Started
```

```
✓ Container grafana Started
```

STEP 4: Verify all containers are running

```
docker compose ps
```



Expected: All 3 services showing "running" status.

STEP 5: Wait 30 seconds for metrics to accumulate, then verify each service.

SECTION 6: VERIFICATION STEPS

6A: Verify ML Metrics Exporter

Open browser: <http://localhost:8000/metrics>

You should see Prometheus-format text output including:

```
inference_requests_total{model_name="sentiment_classifier",...}  
inference_latency_seconds_bucket{...}  
inference_predictions_total{...}  
gpu_utilisation_percent{...}
```

Also check health: <http://localhost:8000/health>

Expected: {"status": "healthy", "model": "sentiment_classifier", "version": "v2.1"}

6B: Verify Prometheus

Open browser: <http://localhost:9090>

1. Click "Status" → "Targets"

Job "ml-inference" should show State: UP

Last Scrape: within last 15s

Job "prometheus" should also show State: UP

2. Go to Graph tab, enter query:

```
rate(inference_requests_total[1m])
```

Click "Execute" → should show a graph with request rate values

3. Try latency query:

```
histogram_quantile(0.99, rate(inference_latency_seconds_bucket[5m]))
```

Should show P99 latency (around 0.08-0.15s normally)

4. Check alerts:

Click "Alerts" → should show 3 defined rules (may be inactive/green)

6C: Verify Grafana



Open browser: <http://localhost:3000>

1. Login with:

Username: admin

Password: admin123

2. Go to Connections → Data Sources

Prometheus should be listed and marked as default

Click it → "Test" button → should say "Data source is working"

3. Explore tab: Select Prometheus, enter query:

`inference_requests_total`

Should show data points

SECTION 7: BUILD THE MONITORING DASHBOARD

7A: Create New Dashboard

1. In Grafana: Click "+" (top-right) → "New Dashboard" → "Add visualisation"
2. Select data source: Prometheus
3. IMPORTANT: In the query editor (bottom of screen), click "Code" button (top-right of the query editor, next to "Builder") — this lets you paste PromQL directly instead of using the visual builder.

7B: Panel 1 — Inference Latency (P50, P95, P99)

Visualisation type: Time series (default — no change needed)

Query A (already shown):

1. In the Code editor, paste:
`histogram_quantile(0.50, rate(inference_latency_seconds_bucket{model_name="sentiment_classifier"}[5m]))`
2. Below the query, expand "Options" → set Legend field to: P50 (plain text, no curly braces)

Query B:

1. Click "+ Add query" (below Query A)
2. Switch to Code mode for this query too
3. Paste:
`histogram_quantile(0.95, rate(inference_latency_seconds_bucket{model_name="sentiment_classifier"}[5m]))`
4. Set Legend to: P95

Query C:

1. Click "+ Add query" again
2. Switch to Code mode
3. Paste:

```
histogram_quantile(0.99,  
  rate(inference_latency_seconds_bucket{model_name="sentiment_classifier"}[5m]))
```
4. Set Legend to: P99

Title:

On the RIGHT sidebar, under "Panel options" → Title field
Change from "Panel Title" to: Inference Latency

Y-axis unit:

On the RIGHT sidebar, expand "Standard options"
Unit: type "seconds" → select "seconds (s)"
This displays "ms" / "s" on the Y-axis instead of raw decimals

SLA threshold (optional but recommended):

On the RIGHT sidebar, expand "Thresholds"
Click "+ Add threshold" → set value to 0.2 → color: red
This draws a red line at 200ms showing the SLA boundary

Save this panel:

Click "Apply" (top-right blue button) — this saves the panel to the dashboard
You'll return to the dashboard view with this panel visible

7C: Panel 2 — Request Throughput (QPS)

1. Click "Add" (top-right of dashboard) → "Visualisation" → Select Prometheus
2. Switch to Code mode in query editor
3. Paste:

```
sum(rate(inference_requests_total{model_name="sentiment_classifier"}[1m]))
```
4. Set Legend to: QPS
5. Right sidebar → Title: Request Throughput
6. Click "Apply"

7D: Panel 3 — Prediction Distribution

1. Click "Add" → "Visualisation" → Select Prometheus
2. Change visualisation type: Top-right dropdown that says "Time series"
→ click it → select "Pie chart"
3. Switch to Code mode in query editor

4. Paste:

```
sum by (predicted_class) (inference_predictions_total{model_name="sentiment_classifier"})
```

5. Set Legend to: {{predicted_class}}

(WITH curly braces here — this references the label value)

6. Right sidebar → Title: Prediction Distribution

7. Click "Apply"

7E: Panel 4 — GPU Utilisation

1. Click "Add" → "Visualisation" → Select Prometheus

2. Change visualisation type: dropdown → select "Gauge"

3. Switch to Code mode

4. Paste:

```
gpu_utilisation_percent{gpu_id="gpu-0"}
```

5. Legend: leave empty

6. Right sidebar → Title: GPU Utilisation

7. Right sidebar → scroll down to "Standard options":

Unit: select Misc → Percent (0-100)

Min: 0

Max: 100

8. Right sidebar → scroll to "Thresholds":

Green: base (0)

Yellow: 70

Red: 90

9. Click "Apply"

7F: Panel 5 — Error Rate

1. Click "Add" → "Visualisation" → Select Prometheus

2. Change visualisation type: dropdown → select "Stat"

3. Switch to Code mode

4. Paste:

```
sum(rate(inference_errors_total{model_name="sentiment_classifier"}[5m])) /  
sum(rate(inference_requests_total{model_name="sentiment_classifier"}[5m])) * 100
```

5. Legend: leave empty

6. Right sidebar → Title: Error Rate

7. Right sidebar → Standard options:

Unit: Misc → Percent (0-100)

Decimals: 2

8. Right sidebar → Thresholds:

Green: base (0)

Yellow: 2

Red: 5



9. Click "Apply"

7G: Save Dashboard

1. Click the disk/save icon (top-right) or press Ctrl+S
2. Dashboard name: "ML Inference Monitoring"
3. Click "Save"
4. Set time range: click "Last 6 hours" (top-right) → change to "Last 15 minutes"
5. Set auto-refresh: click the refresh dropdown (next to time range) → select "10s"
6. Save again (Ctrl+S) to preserve these settings as defaults

SECTION 8: CREATE AN ALERT RULE

1. In Grafana: Go to Alerting → Alert rules → New alert rule
2. Name: "High P99 Latency"
3. Query:
`histogram_quantile(0.99,
rate(inference_latency_seconds_bucket{model_name="sentiment_classifier"}[5m]))`
4. Condition: IS ABOVE 0.2 (200ms)
5. Evaluate every: 1m
For: 2m (must be above threshold for 2 minutes)
6. Labels: severity = warning
7. Save rule
8. To test: The simulated exporter occasionally generates latency spikes.
Wait for a spike or modify app.py to temporarily increase latency.

SECTION 9: USEFUL PROMQL QUERIES REFERENCE

Request rate (QPS):

```
sum(rate(inference_requests_total[5m]))
```

P99 latency:

```
histogram_quantile(0.99, rate(inference_latency_seconds_bucket[5m]))
```

Error rate percentage:

```
sum(rate(inference_errors_total[5m])) / sum(rate(inference_requests_total[5m])) * 100
```



Errors by type:

```
sum by (error_type) (rate(inference_errors_total[5m]))
```

Predictions per class:

```
sum by (predicted_class) (rate(inference_predictions_total[5m]))
```

Average confidence:

```
inference_confidence_avg{model_name="sentiment_classifier"}
```

GPU utilisation:

```
gpu_utilisation_percent
```

Total requests in last hour:

```
increase(inference_requests_total[1h])
```

SECTION 10: CLEANUP

Stop all services:

```
docker compose down
```

Stop and remove volumes (full cleanup):

```
docker compose down -v
```

Remove built images:

```
docker compose down --rmi all -v
```

SECTION 11: TROUBLESHOOTING

ISSUE: "Cannot connect to Docker daemon"

FIX: Start Docker Desktop. Wait for it to fully initialise.

ISSUE: "Port 3000 already in use"

FIX: Find and stop the process:

Windows: `netstat -ano | findstr :3000 → taskkill /PID <pid> /F`

Mac/Linux: `lsof -i :3000 → kill <pid>`

ISSUE: "Prometheus target DOWN"



FIX:

1. Check ml-exporter is running: `docker compose ps`
2. Check logs: `docker compose logs ml-exporter`
3. Test directly: `curl http://localhost:8000/metrics`
4. If networking issue: ensure both on same Docker network

ISSUE: "No data in Grafana panels"

FIX:

1. Verify Prometheus data source: Connections → Data Sources → Test
2. Check time range (top right) covers when services started
3. Try query in Prometheus UI first (localhost:9090)
4. Wait 30-60s after starting for data to accumulate

ISSUE: "Grafana login not working"

FIX: Credentials are admin / admin123 (set in docker-compose.yml)

If changed and forgotten: `docker compose down -v && docker compose up -d`

ISSUE: "ml-exporter build fails"

FIX:

1. Ensure Dockerfile exists in ml-exporter/ folder
2. Check Python syntax in app.py
3. Rebuild: `docker compose build --no-cache ml-exporter`